

M-PATH

Scott Hopkins A01049600
Lina Yang A01167256

Codebase Documentation.....	3
Code Repository.....	3
Tech Stack.....	3
Project Structure.....	3
Services Layer (services/).....	3
Types Layer (types/).....	4
Utils Layer (utils/).....	4
Running Locally.....	4
High Level View of Pages.....	5
Home Page.....	5
Key Features.....	5
Goals Page.....	5
Key Features.....	5
Weekly Summary Page.....	6
Key Features.....	6
Charities Page.....	6
Key Features.....	6
Profile.....	7
Key Features.....	7
Unit Tests.....	8
Test Results:.....	8
Profile (services/profile.test.ts).....	8
updateNoAds.....	8
Supabase (services/supabase.test.ts).....	9
updateCharityPoint.....	9
fetchCharities.....	9
addNewCharity.....	9
Milestones (services/milestones.test.ts).....	10
getCheckInProgress.....	10
Goals (services/goals.test.ts).....	10
createGoal.....	10
updateGoalCompletion.....	10
Goals (utils/goals.test.ts).....	11
mapGoalRecordToHabit.....	11
Notifications(utils.notifications.test.ts).....	11
scheduleDailyGoalReminder.....	11
Week (utils/week.test.ts).....	12
startOfWeekMonday.....	12
addWeeks.....	12
formatWeekRange.....	13
CI/CD Pipeline Configuration.....	14

CI/CD Workflow.....	15
Continuous Integration Pipeline.....	15
Known limitations.....	15
Project Report.....	16
Problem Statement.....	16
Architectural Choices.....	16
Layered Architecture.....	16
Key Technology Decisions.....	16
Diagram.....	17
Key Challenges and Resolutions.....	18
Lessons Learned.....	18
Potential Future Improvements.....	19
Code Review.....	19

Codebase Documentation

Code Repository

The current app is held in a Git Repository following the format of branching and working individually while merging to main, resolving git conflicts when it arises. Merges occur when certain shared aspects need to be updated.

Repo Link: <https://github.com/LinxelLina/COMP7082-HealthCareApp/tree/main/mpath>

Tech Stack

- Expo Go / React Native
- Supabase
- SQLite
- Expo Router
- Expo Notifications
- TypeScript

Project Structure

app/	Main screens and route files
app/(tabs)/	Tab-based screens like Home, Summary, Charities, and Profile
assets/images	App icons, images, and mascot GIFs
components/	Broader reusable UI components
components/ui	Reusable UI components
constants/	Logic for constants such as theme for the Application
hooks/	Events, modified behaviours, and custom functionality
scripts/	Small project scripts
services/	SQLite logic and Supabase logic
types/	Reusable components such as types, interfaces, and records.
utils/	Shared helpers such as Supabase client, notifications calculations

Services Layer (services/)

<https://github.com/LinxelLina/COMP7082-HealthCareApp/tree/main/mpath/services>

All database and API Interactions are abstracted into the services layer so components never interact with Supabase or SQLite directly.

- goals.ts - fetching, creating, updating and deleting goals. Includes cleanup logic for expired goals.
- milestones.ts - formatting and mapping for Milestones
- profile.ts - fetching and updating user profile data including charity selection
- supabase.ts - charity fetching, data for graphs, and charity point updates

Types Layer (types/)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/tree/main/mpath/types>

Shared TypeScript types used across the application.

- category.ts - defines goal category options and records
- habit.ts - defines the Habit type used throughout the goals feature
- goals.ts - form types and prop types for goal components
- charity.ts - charity types, props, and bar chart data types
- profile.ts - defines profile type
- milestones.ts - defines milestone type
- icons.ts - Icon prop type

Utils Layer (utils/)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/tree/main/mpath/utils>

- formatting.ts - date and time formatting functions used in forms
- notification.ts - scheduling daily reminders via Expo Notifications
- week.ts - time related functions for calculating weeks
- goals.ts - mapping and calculation services for goals

Running Locally

Please refer to the README in COMP7082-HealthCareApp/mpath/README.md

https://github.com/LinxellLina/COMP7082-HealthCareApp/tree/dynamic_list/mpath

High Level View of Pages

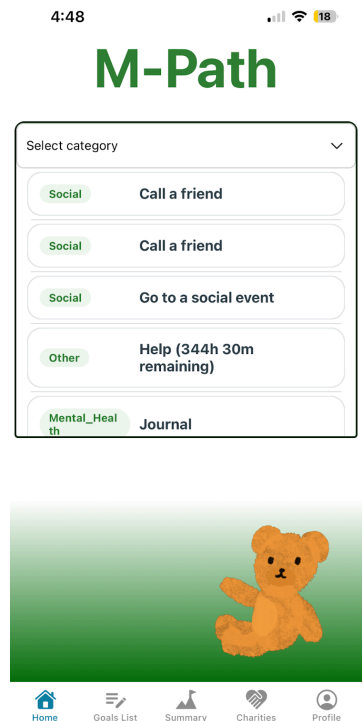
High level view of all the pages currently within the application.

Home Page

The home page serves as the main entry point of the application. It displays the user's personal current goals list and features an interactive mascot (Maurice) at the bottom of the screen that reacts when tapped.

Key Features

- Displays the goals list filtered by category, defaulted to All.
- Interactive mascot with pet animation
- Daily notification handler initialized



Home Page:

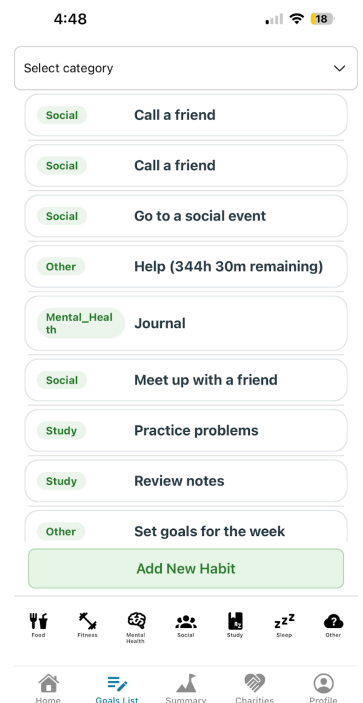
[https://github.com/Linxellina/COMP7082-HealthCareApp/blob/main/mpath/app/\(tabs\)/index.tsx](https://github.com/Linxellina/COMP7082-HealthCareApp/blob/main/mpath/app/(tabs)/index.tsx)

Goals Page

The Goals page serves as the main hub for managing habits and goals. Users can view their full goals list, create new goals, and interact with existing ones directly from this screen.

Key Features

- View all goals in a scrollable list, filterable by category
- Create custom goals via the “Add New Habit” form
- Add pre-made goals from category shortcuts at the bottom of the screen
- Swipe left on a goal to delete it
- Long press a goal to mark it as complete
- Tap a goal to view its full details and milestone progress
- Convert any goal into a milestone from the detail screen

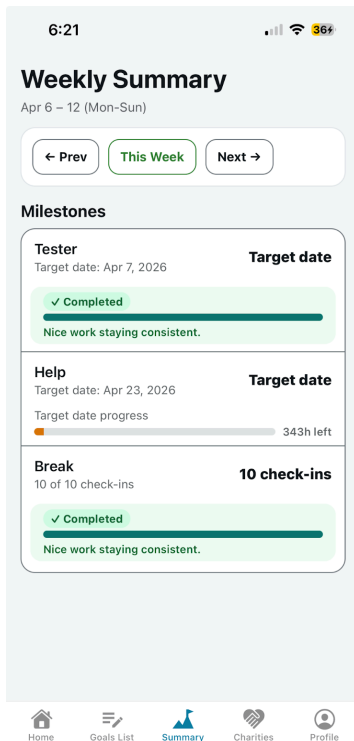


Goal Page:

[https://github.com/Linxellina/COMP7082-HealthCareApp/blob/main/mpath/app/\(tabs\)/goal_page.tsx](https://github.com/Linxellina/COMP7082-HealthCareApp/blob/main/mpath/app/(tabs)/goal_page.tsx)

Weekly Summary Page:

[https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/app/\(tabs\)/summary.tsx](https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/app/(tabs)/summary.tsx)



Weekly Summary Page

The weekly summary page serves as a tracker that allows users to view progress on milestones for the current or past weeks.

Key Features

- Navigate between weeks using previous and next buttons
- View all milestones with their tracking type and current progress
- Progress bars change colour based on completion percentage
- Completed milestones are highlighted green
- Displays current count against target count
- Display time left on target date goals

Charities Page

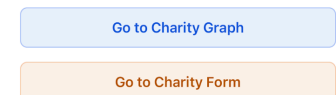
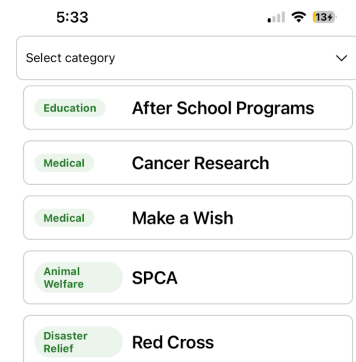
The Charity page displays all charities currently part of the ad-revenue program. Users can browse and filter charities by category, view contribution data, and apply to add a new charity to the program.

Key Features

- Browse all charities in a scrollable list, filtered by category
- Long-press charity to view full details including description, contact, and funds raised
- Tap “Visit Website” within details view to open the charity website in a built-in browser
- View the global contribution graph via the “Go to Charity Graph” button
- Submit a charity application via “Go to Charity Form” button

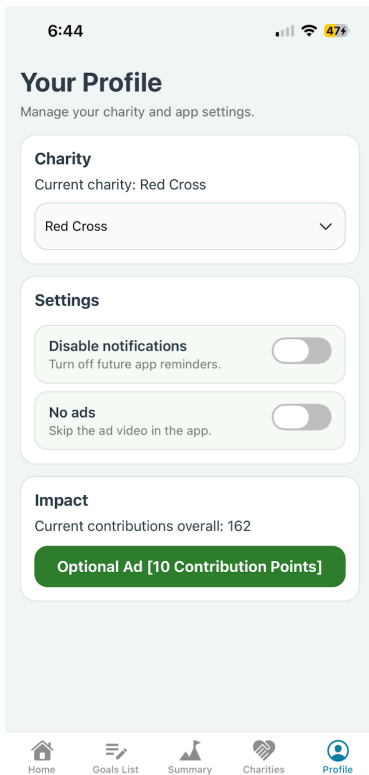
Charity Page:

[https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/app/\(tabs\)/charity_list.tsx](https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/app/(tabs)/charity_list.tsx)



Profile Page:

[https://github.com/LinxelLina/COMP7082-HealthCareApp/blob/main/mpath/app/\(tabs\)/profile.tsx](https://github.com/LinxelLina/COMP7082-HealthCareApp/blob/main/mpath/app/(tabs)/profile.tsx)



Profile

The Profile page allows users to manage their preferences and track their overall charitable impact. Users can select which charity receives their contribution points, manage notification and ad settings, and watch optional ads to generate additional points.

Key Features

- Select which charity to contribute points towards
- Toggle notifications on or off
- Toggle ads on or off
- View total contribution points generated across all charities
- Watch an optional ad via “Optional Ad [10 Contribution Points]” button to earn additional points for the selected charity

Unit Tests

Unit tests are implemented using Jest to verify the core business logic and service functions of the application.

Test Results:

```
> mpath@1.0.0 test
> jest --passWithNoTests

PASS services/supabase.test.ts
PASS services/goals.test.ts
PASS utils/notifications.test.ts
PASS utils/week.test.ts
PASS services/milestones.test.ts
PASS services/profile.test.ts
PASS utils/goals.test.ts

Test Suites: 7 passed, 7 total
Tests:       30 passed, 30 total
Snapshots:   0 total
Time:        1.23 s
Ran all test suites.
```

Profile (services/profile.test.ts)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/services/profile.test.ts>

updateNoAds

Test Case	Input	Expected Output	Result
Stores the no-ads choice as enabled in the local profile database	true	Database updated with no_ads = 1	Pass
Stores the no-ads choice as disabled in the local profile database	false	Database updated with no_ads = 0	Pass

Supabase (services/supabase.test.ts)

<https://github.com/Linxellina/COMP7082-HealthCareApp/blob/main/mpath/services/supabase.test.ts>

updateCharityPoint

Test Case	Input	Expected Output	Result
Does not add a local donation when Supabase update fails	Mocked RPC saying RPC failed	updateCharityPoint() rejects	Pass
		mockRPC called with increment	Pass
		mockedAddDonation to not have been called	Pass
Adds a local donation when the Supabase update succeeds	Mocked RPC with donation of 1	mockRPC called with increment	Pass
		mockedAddDonation called with (1)	Pass

fetchCharities

Test Case	Input	Expected Output	Result
Maps Supabase charity rows into the app charity format	Charity Object	Object in App charity format	Pass

addNewCharity

Test Case	Input	Expected Output	Result
Returns false when the Supabase insert fails	Mocked insert resolved value with error: insert failed	mockFrom to be called with "charity"	Pass
		Return false for failure	Pass

Milestones (services/milestones.test.ts)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/services/milestones.test.ts>

getCheckInProgress

Test Case	Input	Expected Output	Result
Caps progress at 100 when check-ins go past the target	Result of getcheckinprogress when count higher than target	100	Pass
Returns null when a check-in milestone has no target	Result of getcheckinprogress without milestone target	null	Pass
Returns null when the milestone is not a check-in milestone	Result of getcheckinprogress with milestone but without check-in	null	Pass
Returns the expected percentage for a valid check-in milestone	Result of getcheckinprogress when count 50% of target	50	Pass

Goals (services/goals.test.ts)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/services/goals.test.ts>

createGoal

Test Case	Input	Expected Output	Result
Trims the title before saving and returns the inserted id	Result of createGoal() with whitespace in title	Trimmed title and inserted ID	Pass

updateGoalCompletion

Test Case	Input	Expected Output	Result
Stores completed goals as 1 in the local database	Result of await updateGoalCompletion(5, true)	Last call to mockdb is (UPDATE, 1 ,5)	Pass

Stores incomplete goals as 0 in the local database	Result of await updateGoalCompletion(5,false)	Last call to mockdb is (UPDATE, 0 5)	Pass
--	---	--------------------------------------	------

Goals (utils/goals.test.ts)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/utils/goals.test.ts>

mapGoalRecordToHabit

Test Case	Input	Expected Output	Result
Converts a goal record into a habit used by the app	Mocked goal record	Goal record in habit format of app	Pass
Uses fallback values when optional fields are missing	Nearly empty mocked goal record	Habit format record with fallback values like Other and ""	Pass
Converts db flags into booleans and handles goals without duration	Mocked goal record with db flags	Flags converted to booleans	Pass
Uses an empty string id when the goal id is missing	Mocked goal record without goal id	Habit record with id of ""	Pass
Uses created_at as the habit start date	Mocked goal record without habit start date	Habit record with start_date = created_at	Pass

Notifications(utils.notifications.test.ts)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/utils/notifications.test.ts>

scheduleDailyGoalReminder

Test Case	Input	Expected Output	Result
Schedules daily reminder if notifications are allowed	Mocked: profile, permissions allowed, notification scheduled	Reminder fires with expected message	Pass

Returns null and does not schedule when notifications are disabled	Mocked profile with notifications off	Notification functions not called	Pass
Does not request permission if it's already granted	Mocked profile with permissions previously granted	Permissions not called, but notifications called	Pass
Returns null when permission is denied	Mocked profile with permissions denied	Expect permissions to be called, but not notifications	Pass
Schedules after prompting and receiving perms	Mocked profile with notification prompt and permissions	Permissions called, notifications called, notification with correct message	Pass

Week (utils/week.test.ts)

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/mpath/utils/week.test.ts>

startOfWeekMonday

Test Case	Input	Expected Output	Result
Returns the Monday of the same week and resets the time to midnight	Wednesday Mar 6 2024 at 15:30:45	Mon Mar 4 2024 at 00:00:00	Pass
Treats Sunday as part of the week that started on the previous Monday	Sunday Mar 10 2024 at 09:00	Mon Mar 4 2024	Pass

addWeeks

Test Case	Input	Expected Output	Result
Adds whole weeks without mutating the original date	Mar 6 2024, add 2 weeks	Mar 20 2024, original date unchanged	Pass

formatWeekRange

Test Case	Input	Expected Output	Result
Formats week range within the same month	Mon Mar 4 2024	Contains "4", "10", and "-"	Pass
Formats week range crossing into a new month	Mon Mar 30 2024	Contains "30", "5", and "-"	Pass

CI/CD Pipeline Configuration

Located in .github/workflows/ci.yml

<https://github.com/LinxellLina/COMP7082-HealthCareApp/blob/main/.github/workflows/ci.yml>

name: CI

on:

pull_request:

push:

branches:

- main
- scott-dev

jobs:

validate:

runs-on: ubuntu-latest

defaults:

run:

working-directory: mpath

steps:

- name: Check out repository

uses: actions/checkout@v4

- name: Set up Node.js

uses: actions/setup-node@v4

with:

node-version: 20

cache: npm

cache-dependency-path: mpath/package-lock.json

- name: Install dependencies

run: npm ci

- name: Run lint

run: npm run lint

- name: Run tests

run: npm test

CI/CD Workflow

<https://github.com/LinxellLina/COMP7082-HealthCareApp/actions>

✓ readme and updateCharityPoints fix CI #3: Commit 4bb2c31 pushed by scotthops	scott-dev	Today at 9:09 PM 39s	...
✗ milestones tests CI #2: Commit 9d6acd4 pushed by scotthops	scott-dev	Today at 9:00 PM 41s	...
✓ test workflow happens on push CI #1: Commit f2f1316 pushed by scotthops	scott-dev	Today at 8:35 PM 38s	...

Continuous Integration Pipeline

- This project uses GitHub Actions to continuously integrate on every pull request and on pushes to the main branch of the repo.
- The pipeline automatically installs dependencies, runs lint, and runs tests.
- Its purpose is to catch breaks early and improve reliability by validating code automatically before and / or after integration.
- The project prioritised CI over full CD because the repo is still in the prototyping stage. The most meaningful automation at this stage is validating installation, code quality, and test results on each change

Known limitations

- Some charity features depend on a matching Supabase project, tables, and RPC functions already existing
- Time constraint has caused some features to be less built out then expected, will consider it a “feature” in the development process.
- Code is organized but not documented with JSDoc format, but is easy to follow and names are self-explanatory.
- Due to time constraints test cases are limited to core functionality.
 - Please read the README for more information:
<https://github.com/LinxellLina/COMP7082-HealthCareApp/tree/main/mpath#testing-approach>

Project Report

Problem Statement

Many people generally find it hard to build small consistent healthy habits and lack the motivation to maintain them over time. MPath addresses this by making small, healthy actions feel more visible and meaningful. Allowing the users to set goals, track milestones, and donate to charities of their choice for completing habits. Celebrating the small goals and connecting the user to a global community as interactions with the app build towards revenue for charities.

Architectural Choices

Layered Architecture

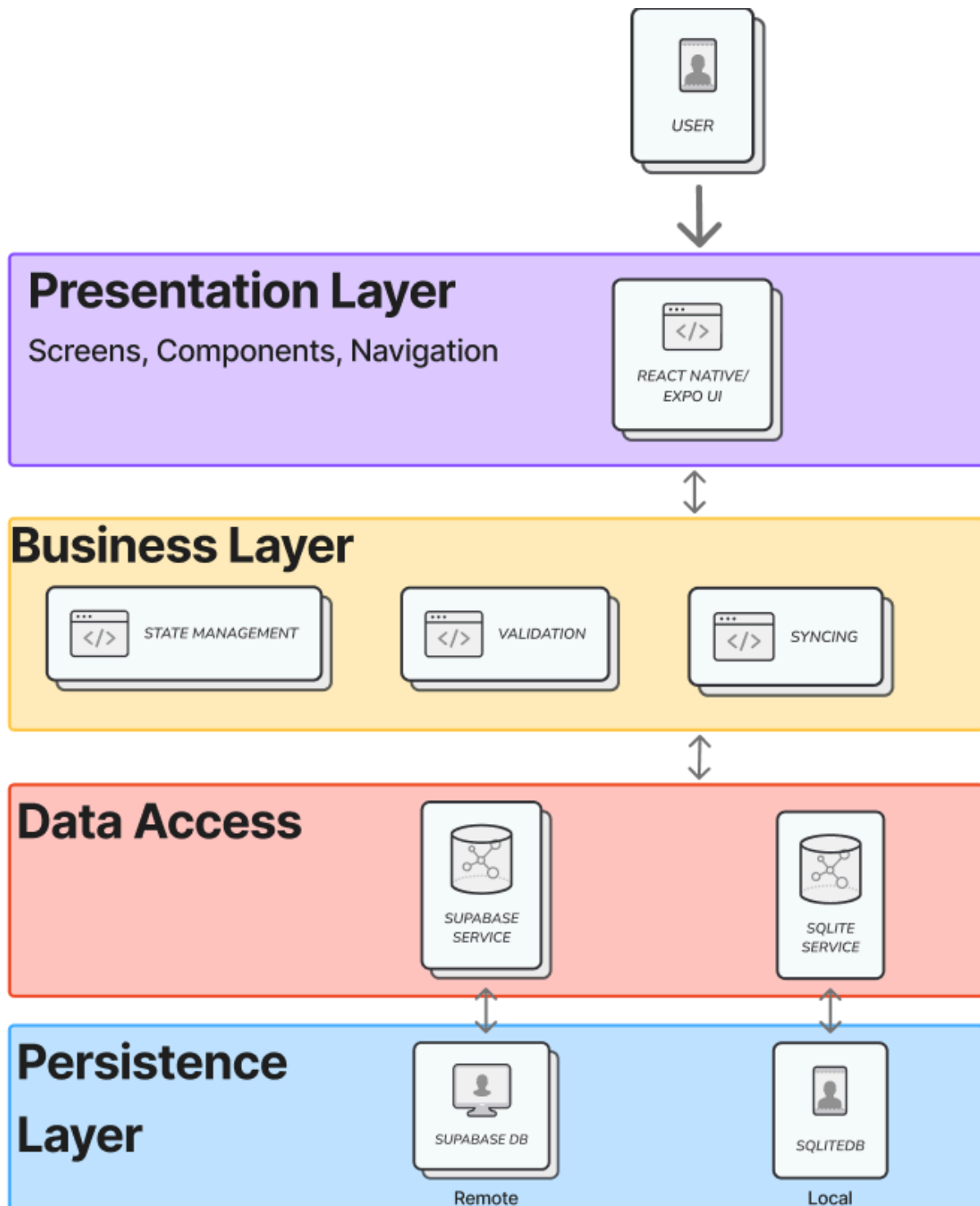
The application follows a four-layer architecture:

- Presentation - React components built with Expo Go
- Business Logic - Custom hooks and state management
- Data Access - Service layer abstracting Supabase and SQLite calls
- Persistence - Supabase (remote PostgreSQL) and SQLite (on-device)

Key Technology Decisions

- Expo Go / React Native - Cross-platform (iOS & Android) mobile development
- Supabase - Remote database and authentication
- SQLite - Offline-first local storage
- Expo Router - File-based navigation

Diagram



Key Challenges and Resolutions

- Nested Views of different types causing cooperation issues for the program.
 - Removed Views that were not necessary, breaking each screen issue down and removing and editing nested views to reduce code breaks.
 - Iteratively building the pages to avoid View conflicts.
- Keeping all user goals on a single database causing issues for scalability and identification
 - Separated goals from the Supabase Postgres Database and moved to an offline, local storage database to improve online storage capabilities.
 - Global requirements such as shared Charities remain in Supabase, individual goals on local storage.
- Code deprecation occurring during on-going development.
 - Followed the deprecation messages to the new component and adjusted code accordingly.
- App testing with Expo Go running into issues depending on the local environment.
 - Created and tested backups to ensure testing will run smoothly regardless of environment.
- Shared iOS and Android UI elements not working as expected.
 - Tested and verified new components to replace the UI element that was not working as expected, ensuring better cross-compatibility.

Lessons Learned

- Time and planning is essential for the success of the project and being able to complete tasks by and before the specified time. Plan early, plan ahead, and be generous.
- Communication is key to ensure all group members (2) are on the same page.
- Finalize Database schemas early on into the project and discuss potential expansion problems that could occur.
- Design a wireframe earlier into the start of the application to avoid constant adjustments in UI later on.
- Keeping types, interfaces, and reused functions in separate files to be shared amongst all files to keep things organized and separated for the single responsibility clause.
- Error handling should've been better implemented as the repository became more and more filled out.

Potential Future Improvements

- Multiple design patterns for the user to choose from (including one for accessibility and being ACA compliant).
- Point generation based on goals and milestones completed to be used towards customizability, separate from the charity points.
 - Mascot decoration or character creation option.
- Proper ad revenue design to be implemented into the system.
 - Analytics and measurements of ads and the revenue generated and how to supply that to the charity chain.
- AI or research pool designed to respond to milestones explaining the health benefits of the milestone created.
- Better implementation and storage design for milestones.
- Better graph implementation and layout for global charity contributions.
- Charity form verification process.
- Allowing users to share their progress.

Code Review

Due to the fact there were only two of us, we did not do many pull requests and code reviews. We trusted each other to have tested and when we did merge up it was in-person. Issues were discussed in person leaving github empty of a paper trail. In the last major updates, as we were merging apart, we did pull requests.

<https://github.com/LinxellLina/COMP7082-HealthCareApp/actions>

Pull Request	Reviewer	Changes	Outcome
https://github.com/LinxellLina/COMP7082-HealthCareApp/pull/1/changes	Scott	README update. Jest test and commits. Refactored Goals list. Refactored Charity list. Removed deprecations. Added error handling to Profiles page and components. Updated Supabase for separation of layers.	Approved
https://github.com/LinxellLina/COMP7082-HealthCareApp/pull/2	Scott	Merged Scotts branch to Lina's. Refactored Milestones. Test cases added by Scott.	Approved
https://github.com/LinxellLina/COMP7082-HealthCareApp/pull/3	Lina	Milestone tests created Edited Configuration yaml file	Approved

https://github.com/LinxellLin/COMP7082-HealthCareApp/pull/4	Lina	Profile and milestone tests Goals and Supabase tests	Approved
---	------	---	----------